機械学習に関する議論

3d5d8d

2025/11/02

これは, 畳み込みニューラルネットの解析に関する議論の備忘録である. 本稿は議論の流れに沿って進められるので, 可読性は非常に低いことに留意してほしい. この拙文を諸君らが読んだとしても恐らくは何も得られない.

これは明日の, 今の私ではない私におくるものなのだから.

1 Fundementals of Machine Learning

本稿は, ゼロから作るディープラーニング程度の知識を前提としているが, 重要な概念は繰り返し説明する. まず, 我々は機械学習の中でもニューラルネットワーク（NN）について探求してきた. NN とは端的に言えば, テストデータを正しく分類するために最適な重み行列とバイアス行列（これが所謂 weight）を求める回帰アルゴリズムである. 最適, の指標は損失関数（criterion）であり, その損失関数が最小となるように学習を進めることが最適化の意味するところである. これは我々が学習済みのモデルを構築するとき, つまりは良いモデルを構築するときに頭に入れておくべき事柄なのだ. しかし, 私達が自身で学習させたわけではないモデルを活用しようと思ったとき, その行列がなんのパラメータに対し最適化されているかは, 必ずしも自明ではない.

そんなときに役立つのが, 以下のコードである.

Listing 1 hoge

```
params = [p for p in model.parameters()]
```

我々は主に MNIST と呼ばれる数字の画像群をトレーニングデータ, テストデータとして基本的に用いる. なぜならばこれが最も簡単に扱えて, 論文に用いられているデータセットであるからである. 例えば後にでてくるが, resnet34 というモデルは ImageNet1K と呼ばれるトレーニングデータの中に含まれるパラメータに対し最適化されている. MNIST は 1 チャンネルのグレースケール画像であり, Imagenet は 3 チャンネル（赤青緑）のカラー画像であるから, テストデータとして MNIST を使う場合には前処理が必要となる. 詳細に関しては, pytorch の web サイトにあるから参考にしてほしい.

> It is not surprising that using resnet34 on mnist data that you get very low accuracy. This is because the pretrained model that you downloaded was not trained on mnist. Actually it is even strange that you could use it on mnist, because mnist is a 1 channel dataset (grayscale) whereas imagenet is a 3 channel (red-green-blue) data

トレーニングデータからモデルを得るときには, 我々は確率的勾配降下法と呼ばれるアルゴリズムをよく用いる. これにもいくつか種類があるが, 今のトレンドは Adams なので, 私もそれに倣っている. 些細なことは

おいておいて, 非常に直感的に仕組みを説明していく.

トレーニングデータは無数のパラメータを持っている. そしてパラメータの収納された行列全体に対して, 全体の勾配ベクトルを下げる方向に学習は進む. これはパラメータ空間において, 損失関数によって構築される複雑な地形（losslandscape）の谷底を見つけることを意味する. その地形を探索する旅人は自身の進む方向が上りなのか下りなのかしか判断することができず, ただそれだけを手がかりに谷底, つまりは損失関数の勾配ベクトルが 0 になるところを探す.

他の表現方法もあるかもしれない. これは XOR の分割問題として考えることができるのだ.

> The NN applies many affine transformations to an input and in between the affine transformations applies nonlinear functions at each coordinate. In the end we want to predict a relationship between the input and the label. The reason one uses cross entropy loss is because you are trying to fit to some categorical label. The categorical label can be represented as a probability distribution on K points (if there are a total of K classes). This means that at the end of the neural network you should output also probabilities. And this you achieve by that softmax layer. I understand the sequence: softmax makes outputs(z) into lie in (0,1) and sum to 1. we can treat it as probability distribution y. Finally, using cross-entropy, we evaluate loss y to t. And, we should select a suitable loss function with respect to saturation (vanishing gradients) and learning speed. Also I believe the reasons we use cross-entropy function is making gradient calculation simple (like (y-t) = error) in the backpropagation process. Notice that you took a gradient of the loss with respect to the logits (the z’ s, i.e. before the softmax) and you got the difference between the target (which in most applications is a one-hot-encoded vector) and the probabilities y (after the softmax has been applied to the logits).

1.1 歩幅, 摂動について

ここでの議論は以下の url に記載した Welch Labo の動画, github の内容も参考にしている.

`https://www.youtube.com/watch?v=NrO20Jb-hy0`

`https://github.com/stephencwelch/ai_book/blob/main/2_wormhole.ipynb`

最適な weight を見つけてくれるのは, 自分がどこにいるかさえわからない旅人である. 足元の角度, 勾配しか頼りにできない以上, 彼らは自身が浅い谷底にいたとしても気づくことができない. ときに我々は彼らの到着地点の景色をみたい. なぜならそれがモデルの学習がうまく行っているか, つまりは losslandscape を緻密に探索できていて, 局所的な谷にハマっていないかを知る重要な鍵であるからだ. しかし彷徨う losslandscape は, つまりはパラメータ空間は一般に多次元であるから, 可視化するのは非常に困難である.

そこで我々は微小な摂動 (perturbation) という概念を取り入れることにしよう. 自身で定めた epoch 数分だけミニバッチの学習が終わったら, 学習済みモデルが手に入る. この最適なパラメータ行列は, 可視化した losslandscape 上において $\theta = 0$ となる. 多次元のパラメータ空間を, ある方向のベクトル v でスライスすることで, 我々は地形の断面図を見ることができる. ここで我々は 2 つの検討すべき事項に直面する. 1 つは, ベクトル v をどのように設計するかである. ランダムな勾配を選ぶのか, それともパラメータ勾配が減少するようなベクトル方向として切断するのかは検討の余地がある. 2 つ目は, その perturbation をどの程度与えるかである. 最適なパラメータ $\theta = 0$ から, 摂動分, つまりはほんの少しだけずらして関数値がどうなるか調べプ

ロットすることで, 我々はその方向の断面図を得ることができる. 例えば v をランダムなベクトルとするならば, そのノルムは私の環境では約 700 である. このことは簡単に確認でき, まず生成したランダムベクトルの次元 (つまりは θ の次元) を確認する.(私の場合は 43 万であった) すべての成分に対して, normalization されているものとすれば, その平方根がノルムとなる. 単に摂動ベクトルとして θ に v を足しだけでは, $\theta = 0$ と $\theta + v$ のときの 2 点の損失しかわからず, 調べる価値がない. そこで, 摂動のノルムを十分細かく分割して, 係数 t を適切に与えて少しずつ加える摂動を変化させていけば, 意味あるプロットが得られるとわかる. すなわち, $\theta + t \cdot v$ を計算すればよい. t は linspace コマンドで例えば以下のように指定する. 第 1 引数が t の正方向, 第 2 引数が t の負方向, 第 3 引数はプロット点数である.

Listing 2　hoge

```
def analyze_loss_landscape(model, test_loader, criterion):
    # 訓練済みパラメータを辞書形式で保存
    trained_params = {name: param.data.clone() for name, param in model.
        named_parameters()}

    # create random perturbation vectors
    random_vector = [torch.randn_like(param.data) for param in model.parameters()]

    # 各層の方向ベクトルを、対応する重み（パラメータ）のノルムで正規化する
    # これが簡易版の Filter-wise Normalization です
    for i, param in enumerate(model.parameters()):
        # ゼロ除算を避ける
        if torch.norm(param.data) > 1e-8 and torch.norm(random_vector[i]) > 1e-8:
            random_vector[i] = random_vector[i] * (torch.norm(param.data) / torch.
                norm(random_vector[i]))

    # tの範囲をより適切に修正
    t_range = torch.linspace(-0.01, 0.01, 100)
    loss_values = []
    for t in tqdm(t_range):
        loss = compute_loss_at_point(model, t, trained_params, random_vector,
            test_loader, criterion)
        loss_values.append(loss)

    return t_range, loss_values
```

このとき t や, 分割数 (プロット数) はどれだけの細かさであれば十分であろうか. 例えば

> linspace(-1, 1, 50) means that it divides the range max(700 × 1)into 50 points. Of course, this is too rough.

故に, 大体は t を 0.01 程度, プロット数が 100 点程度になるようにするのが推奨される.

次に perturbation ベクトル v を, 勾配方向に合わせるプログラムは以下のようにかける.

> Maybe try to plot the loss landscape on the same graph for many random vectors. And also play with the range of t. Make it a more refined mesh and also longer. Then later try to change the random vector to the gradient of the loss for a particular data point.

Listing 3　hoge

```
meaning vec = torch.autograd.grad(criterion(model(x), x)) for some single
    image x.
See if this makes a difference.
```

1.2　epoch 数, バッチサイズについての考察

ここではエポック数やバッチサイズについて考察する．エポック数が十分でないとき, 良い landscape の画像は得られない．私の環境では 10 程度では足らず, 30 程度であれば良い結果が得られた．

1.3　learning rate を代表とするハイパーパラメータの設計

1.4　Normalization に関する重要な議論

なぜ,

1.5　平坦性に関する検討（2025/11〜）

我々はこれまでの試みから, Loss landscape を比較的正確に描くことができるようになった．摂動ベクトルの正規化は, パラメータの大きさに対し十分意味のあるノルムをもった摂動を加えるうえで, 非常に重要な役割を担う．Loss landscape を見る際には, y 軸のスケールをよく見て, 必要に応じて linscape のレンジを広げてやれば, 多くの場合複雑な地形を見ることができる．

ここからは, loss landscape の谷底の形状を調べることに興味がある．端的に言えば, 損失関数を最小にするような最適化パラメータから, パラメータがずれたときにどれだけ損失が大きくなるかを調べたいというモチベーションである．谷底が平坦であればあるほど, 汎化性能が高いと我々はよび, モデルのロバストネスを調べるうえではこの考え方が重要である．